Capgemini CV and Codebase Q&A Handbook

Direct, code-backed answers for technical, project, cloud, coding, DevOps, and HR interviews

Prepared from the supplied CV and repositories. Python is the primary coding language for solutions. The handbook gives answers you can rehearse, then separates what the code proves from the production improvement that demonstrates mature engineering judgment.

Important: do not claim features the source does not implement. For example, the AI repository does not use Firebase Authentication or Firestore persistence, and the supermarket repository does not use the Java Logging API or password hashing.

Table of contents

- 1. Interview intelligence and study strategy
- 2. GCP AI Code Reviewer - code-backed Q&A
- 3. Supermarket Billing and Inventory System - code-backed Q&A
- 4. Java, OOP, JDBC, DBMS, and SQL fundamentals
- 5. Python-first coding, DSA, and complexity
- 6. C, C++, JavaScript, HTML, CSS, Tailwind, and API fundamentals
- 7. Cloud, Firebase, Docker, Git, GitHub, AWS, and DevOps
- 8. OS, networking, DBMS, and out-of-nowhere fundamentals
- 9. HR, behavioral, and mock-interview practice
- 10. Coverage matrix and sources

1. Interview intelligence and study strategy

This section reports a preparation signal, not an official Capgemini syllabus. Recent 2026-27 candidate reports repeatedly describe a combined technical and HR discussion lasting about 20-40 minutes. The recurring topics are projects, technologies stated on the resume, OOP, SQL/DBMS, basic code/logic, communication, relocation, and teamwork.

Priority	What to prepare	Reason
P0	Both project stories and exact personal contribution	The resume is commonly used as the interview script.
P0	Java, MySQL, JDBC, OOP, SQL joins, transactions	They are the strongest code-backed items on this CV.
P1	Gemini, Firebase Functions, GCP secrets, Node.js, REST	The AI project has direct implementation evidence.
P1	Python coding basics, DSA, OS, networks	Frequent fresher follow-ups and assessment topics.
P2	AWS, Docker, Tailwind, CI/CD	Present on the CV but not evidenced in the supplied repositories.

1. What should I prioritize the night before the interview?

Direct answer: First, rehearse a 90-second introduction and a 2-minute explanation of each project. Then write one SQL join, one GROUP BY and HAVING query, palindrome/anagram code in Python, the four OOP pillars, JDBC transaction flow, IAM and secret handling, and why Capgemini. This gives the highest return because it connects directly to the resume.

2. How should I answer a question when I do not know the exact answer?

Direct answer: Say what you know, state the boundary, and explain how you would verify it. For example: 'I have not implemented that directly. In my project I used a Firebase HTTPS function; I would check the official documentation and test the behavior in a staging environment before using it in production.' This demonstrates honesty, engineering judgment, and a learning mindset.

3. How do I avoid sounding memorized?

Direct answer: Use a consistent structure but change the wording naturally: problem, user flow, technology choice, your contribution, one challenge, one improvement. Lead with the conclusion, then give one concrete example from the code. Avoid listing tools without explaining why they were needed.

2. GCP AI Code Reviewer - code-backed Q&A

Repository evidence: Firebase Hosting serves the static frontend. `firebase.json` rewrites `/reviewCode` to the `reviewCode` function. The backend is a Firebase Functions v2 HTTPS `onRequest` function. It receives `req.body.code`, retrieves `GEMINI_API_KEY` through `defineSecret`, calls `gemini-2.5-flash-lite` using `@google/generative-ai`, strips Markdown fences, and sends the model text with JSON content type.

Architecture you should draw

```
Browser UI (HTML/CSS/JavaScript)
  -> POST /reviewCode with { code }
  -> Firebase Hosting rewrite
  -> Firebase Functions v2 HTTPS onRequest handler
  -> Firebase Secret parameter: GEMINI_API_KEY
  -> Gemini 2.5 Flash Lite generateContent
  -> text expected to be JSON
  -> browser displays improvedCode and issuesAndFixes
```

4. Explain the AI Code Reviewer project end to end.

Direct answer: I built a cloud-hosted code-review web application. A user pastes source code into a browser UI and submits it to `/reviewCode` as JSON. Firebase Hosting routes that path to a second-generation Firebase HTTPS function. The function retrieves the Gemini API key from a Firebase secret, sends the code with instructions to return issues, fixes, and improved code, then returns the model response to the frontend. The frontend separates `improvedCode` from `issuesAndFixes` and displays both to the user.

Current code truth: The repository proves this exact flow. It does not show user authentication, persistent review history, Firestore reads/writes, a Dockerfile, or a CI/CD pipeline.

Production bar: For production, I would validate the method, content type, body shape, maximum code size, and model output. I would add structured output schema validation, authentication if the service is not public, rate limiting, logging, monitoring, and safe HTML rendering.

5. Why is the backend implemented as a Firebase Functions v2 HTTPS function?

Direct answer: An HTTPS function is suitable because it exposes a small serverless API without managing a virtual machine. It keeps the Gemini key on the server, receives the browser request, makes the model call, and can scale independently of the static frontend. Functions v2 uses the `onRequest` handler for ordinary HTTP requests.

Current code truth: The code imports `onRequest` from `firebase-functions/v2/https`; it is an ordinary HTTP function, not an `onCall` callable function.

Production bar: I would configure region, timeout, memory, concurrency, CORS policy, and observability explicitly. I would use `onCall` only if I wanted Firebase SDK callable semantics and automatic token handling.

6. How is the Gemini API key protected?

Direct answer: The key is declared with Firebase `defineSecret` and attached to the function through the `secrets` option. At runtime the handler gets the secret value rather than embedding it in browser code. That separation prevents every frontend user from receiving the API key.

Current code truth: The source uses `defineSecret('GEMINI_API_KEY')` and has an emulator-only branch that reads `process.env.GEMINI_API_KEY` locally.

Production bar: In production I would rotate the secret, grant access using least-privilege IAM, avoid logging it, and make sure `.env` files and service-account credentials are excluded from Git. Secrets must never be shipped in a frontend bundle or committed to GitHub.

7. What exactly does the prompt ask Gemini to return?

Direct answer: The prompt asks Gemini to review the submitted code and return a JSON object with two string fields: `issuesAndFixes` for bugs, security, and performance discussion, and `improvedCode` for revised code without Markdown fences. This creates a predictable UI contract instead of rendering one unstructured paragraph.

Current code truth: The code requests JSON in prompt text and removes ```json fences with string replacement. It does not parse JSON or validate the response before sending it.

Production bar: A production version should use the current Gemini structured-output facility with a JSON schema, parse the returned JSON, validate field types and length, and reject malformed output. Schema-valid JSON still needs application-level validation because syntactic validity does not prove the recommendation is correct.

8. What is the difference between a prompt asking for JSON and structured output?

Direct answer: A prompt asking for JSON is an instruction that the model can sometimes ignore or wrap in prose. Structured output config supplies a schema and asks the model to conform to it, making parsing more reliable. Even then, I would validate the parsed object because a well-formed response can still contain poor or unsafe recommendations.

9. What is an LLM hallucination in a code reviewer?

Direct answer: It is feedback that sounds plausible but is not supported by the submitted code, such as inventing a dependency, file, vulnerability, or execution path. I would present model output as a review suggestion, ask the model to cite the relevant code, and require human review before a developer accepts a change.

10. How would you improve the model prompt?

Direct answer: I would tell the model the language, code context, desired review categories, severity labels, and the exact response schema. I would ask it to cite the affected snippet or line range, distinguish facts from assumptions, and return 'insufficient context' when the code does not prove a claim. I would also limit input size and separate system instructions from untrusted user code.

11. How does the frontend call the API?

Direct answer: The browser uses `fetch('/reviewCode', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ code }) })`. It disables the review and clear buttons while the request is pending, shows a status message, parses the response as JSON, and restores the controls in `finally`.

Current code truth: The frontend assumes every success response is valid JSON. If the function returns non-JSON text or an error page, `res.json()` throws and the generic error UI is shown.

Production bar: I would check `res.ok`, provide an error body with a stable schema, set timeouts/retry rules appropriate to idempotency, and give users a useful error code or request ID for support.

12. What security issue exists in the frontend response rendering?

Direct answer: The UI passes `issuesAndFixes` through `marked.parse` and assigns the result to `innerHTML`. If the model output contains malicious HTML or a crafted payload, that can create an XSS risk. Treating model output as trusted HTML is unsafe.

Current code truth: The source imports Marked from a CDN and uses `issuesResultEl.innerHTML = marked.parse(data.issuesAndFixes)` without sanitization.

Production bar: I would sanitize the generated HTML with a well-maintained sanitizer or render it as plain text. I would also add a restrictive Content Security Policy and avoid executing any model-generated script or event handler.

13. Does the project use Firebase Authentication or Firestore?

Direct answer: No. The repository contains Firestore rules and empty index configuration, but the source has no Firebase client SDK, no Auth flow, and no Firestore read or write. I would not claim authentication, user history, or database persistence in an interview.

Current code truth: The Firestore rule is also time-limited and expired on 2026-04-11. Since the app does not use Firestore, it should be removed or replaced with secure rules before any future use.

Production bar: If I add review history, I would authenticate users, use Firestore rules that authorize only the owner of each review document, define indexes only for actual queries, and test rules with the Firebase emulator.

14. Does the project use Vertex AI?

Direct answer: The code uses the `@google/generative-ai` package and `GoogleGenerativeAI`, which is the Gemini Developer API SDK pattern. Although `@google-cloud/vertexai` is listed as a dependency and the README says 'Vertex AI / Gemini API', there is no Vertex AI import or call in the implementation. I would describe it as Gemini API integration through a Firebase Function unless I migrate it to Vertex AI.

15. How would you make the endpoint production-ready?

Direct answer: I would allow only POST, validate JSON and code length, cap request size, set a strict CORS allowlist, authenticate or apply App Check where appropriate, rate-limit per identity/IP, add timeouts, return structured errors, sanitize rendered output, log safe metadata, monitor latency and failures, and protect the secret through IAM. I would also use a schema for model output and write unit plus integration tests.

16. How would you scale and control cost?

Direct answer: Serverless functions can scale with incoming requests, but model inference is the dominant cost and latency. I would set maximum instances and concurrency according to safe downstream limits, cap code size and response tokens, rate-limit requests, cache only when safe, monitor token and error metrics, and introduce authenticated quotas for expensive usage.

17. What is Firebase Hosting's rewrite doing?

Direct answer: The hosting configuration maps requests whose source is `/reviewCode` to the `reviewCode` Cloud Function. This lets the browser call a same-origin path instead of hard-coding a separate backend URL, simplifying the frontend and deployment routing.

18. What is the difference between an HTTP function and a callable function?

Direct answer: An HTTP function is a general request handler that I invoke using standard HTTP, as this project does with `fetch`. A callable function uses Firebase client SDK conventions and can automatically carry Firebase Auth and App Check tokens. I would choose HTTP for a plain REST-like endpoint and callable functions when I want Firebase SDK protocol behavior.

19. Why use Node.js for this backend?

Direct answer: Node.js is well suited to a small JSON API because the workload is mostly I/O: receive an HTTP request, call the Gemini service, and return a response. JavaScript lets the browser and backend share a language, and `async/await` makes the asynchronous model call readable. CPU-heavy tasks would need careful design because Node's event loop should not be blocked.

20. What is Docker, and did you use it here?

Direct answer: Docker packages an application and its runtime dependencies into an image so it runs consistently across environments. I should not claim Docker deployment for this repository because no Dockerfile or container configuration is present. If I containerized it, I would use a minimal Node base image, keep secrets out of the image, run as a non-root user, and scan dependencies.

21. What DevOps flow would you add?

Direct answer: On every pull request, GitHub Actions should install dependencies, run linting and unit tests, scan dependencies, and build the function. On an approved main-branch merge, it should deploy to a non-production Firebase project, run smoke tests, require approval for production, deploy with least-privilege credentials, and publish monitoring/rollback information. The supplied repository currently has no CI/CD workflow, so this is a recommended flow, not a current feature.

AI reviewer follow-up quick answers

22. What is REST?

Direct answer: REST is an architectural style that treats resources through standard HTTP methods and representations. This project exposes an action-style review endpoint over HTTP; it should still use clear JSON contracts, status codes, and idempotency reasoning.

23. Why POST for code review?

Direct answer: The request submits potentially large code and triggers a new inference operation, so POST is appropriate. I would not put code in a URL query string.

24. What is IAM?

Direct answer: Identity and Access Management controls who or what can access cloud resources and which actions are allowed. The production principle is least privilege: give the runtime service account only the permissions required.

25. What should be logged?

Direct answer: Log request ID, time, status, latency, model name, input/output sizes, and sanitized error type. Do not log secrets or full proprietary customer code by default.

26. How would you test it?

Direct answer: Unit-test validation, prompt construction, JSON parsing, and errors with the Gemini client mocked. Integration-test the deployed endpoint with a non-production key and representative safe snippets. Manually verify the UI state for empty input, failure, timeout, and valid review.

27. How do you handle model failure?

Direct answer: Catch the exception, log safe diagnostic context, return a stable 5xx or upstream-dependency error response, and tell the user the review could not be generated. Retry only transient failures and only with bounded attempts.

3. Supermarket Billing and Inventory System - code-backed Q&A

The Java repository contains Swing windows for login, admin/cashier dashboards, product CRUD, billing, sales history, and log viewing. It uses MySQL through JDBC. The inferred tables are `users`, `products`, `sales`, `sale_items`, and `logs`. No DDL file is included, so primary keys and column types below are inferred from the application queries.

Architecture and data flow

```
LoginForm (Swing)
  -> DBConnection.getConnection()
  -> users table lookup with PreparedStatement
  -> AdminDashboard or CashierDashboard

ProductManagement -> products CRUD
BillingForm -> cart JTable -> transaction -> sales + sale_items + products stock update
SalesHistory -> sales list -> JOIN sale_items to products
LoggerUtil -> logs table -> ViewLogs
```

28. Explain the Supermarket Billing and Inventory System end to end.

Direct answer: I built a Java Swing desktop application backed by MySQL for authentication, product management, billing, inventory updates, sales history, and activity logs. The login screen verifies the user role and opens an admin or cashier dashboard. Admin users can open product management, sales history, and logs. Cashiers create a cart, the application calculates totals, then persists the sale and its line items and updates product quantities in one JDBC transaction.

Current code truth: The repository proves the GUI screens, JDBC access, prepared statements for most variable queries, sales batches, generated sale IDs, and log-table writes. It does not prove password hashing, strong server-side role enforcement, stock availability checks, or a rollback call.

Production bar: In a production version, I would separate UI, service, and data-access layers; store passwords as salted adaptive hashes; enforce authorization in every privileged operation; validate stock atomically; roll back every failed transaction; use a connection pool; and add automated tests.

29. How does authentication work in the current code?

Direct answer: The login form reads a username and password from Swing fields, opens a JDBC connection, and executes `SELECT * FROM users WHERE username = ? AND password = ?` through a `PreparedStatement`. If it receives a row, it reads the `role` column and opens the matching dashboard.

Current code truth: Using `PreparedStatement` is good because it parameterizes input and reduces SQL injection risk. However, the query compares plaintext passwords and `new String(passwordField.getPassword())` leaves an immutable password

copy in memory.

Production bar: Production authentication should store a salted adaptive password hash such as Argon2id, bcrypt, or PBKDF2. The application should query by username, verify the submitted password against the stored hash in application code, clear the password char array promptly, use generic failure messages, rate-limit attempts, and protect credentials in a secret store.

30. Why use PreparedStatement instead of Statement?

Direct answer: PreparedStatement separates SQL structure from input values. It protects against SQL injection, handles type binding clearly, can be reused, and makes code more maintainable. In this project it is used for login, product CRUD, sale details, and logging. A plain Statement is used only for a fixed sales-history query with no user input, but I would still standardize on PreparedStatement for consistency.

31. How does product management work?

Direct answer: The `ProductManagement` Swing frame loads rows from `products` into a `DefaultTableModel`. It validates non-empty fields, parses price and quantity, and uses parameterized INSERT, UPDATE, and DELETE statements. Selecting a table row fills the form for update or delete, and the grid refreshes after a successful write.

Current code truth: The implementation validates presence and numeric parsing but does not reject negative price or quantity in product CRUD.

Production bar: I would add database constraints such as `price >= 0` and `quantity >= 0`, validate them in the UI/service layer, use decimal currency rather than `double`, handle duplicate product policy, and log product changes with the acting user and old/new values.

32. Explain the billing transaction flow.

Direct answer: The cashier adds products to an in-memory Swing table. On finalization, the code disables auto-commit, inserts a parent row in `sales`, retrieves the generated sale ID, batch-inserts rows into `sale_items`, batch-updates product quantities, and commits the connection. The transaction is important because the sale, its line items, and its stock changes must succeed or fail together.

Current code truth: The code does call `conn.setAutoCommit(false)` and `conn.commit()`, uses `RETURN_GENERATED_KEYS`, and batches sale-item and stock statements. However, the catch block does not explicitly call `rollback()`, and the update can deduct stock below zero.

Production bar: Production code should validate every product and requested quantity in the same transaction, execute a conditional update such as `UPDATE products SET quantity = quantity - ? WHERE id = ? AND quantity >= ?`, verify every update count, roll back on any exception, restore auto-commit if reusing a connection, and report a clear out-of-stock error.

33. Why do you need a transaction for a sale?

Direct answer: A sale is one business event made of multiple database writes: the sales header, every sale item, and every inventory update. Without a transaction, a crash could save the sale header but not the item rows or reduce stock without recording a sale. A transaction provides atomicity: either all related changes commit or none do.

34. What are ACID properties in this project?

Direct answer: Atomicity means a complete sale is saved or none of it is. Consistency means constraints such as valid product references and non-negative inventory remain true. Isolation means concurrent cashiers do not corrupt the same stock record. Durability means committed sales survive a crash. In the current code, atomicity is intended but should be strengthened with explicit rollback and row-count checks.

35. How would you prevent two cashiers from overselling the last item?

Direct answer: I would make stock deduction conditional inside the transaction: update only when the available quantity is at least the requested quantity, then check that exactly one row changed. If the count is zero, I would roll back the whole sale and tell the cashier the item is out of stock. The database is the source of truth; checking stock only in the GUI is not safe when two users act at once.

36. Why does BillingForm use generated keys?

Direct answer: The `sales` table generates an ID after inserting the sale header. The application needs that ID as the foreign key for every `sale_items` row. `Statement.RETURN_GENERATED_KEYS` lets JDBC obtain it without guessing the latest row or running an unsafe separate query.

37. How does the sales-history join work?

Direct answer: The selected sale ID filters `sale_items`, and the query joins `sale_items si` to `products p` using `si.product_id = p.id`. It selects product ID, product name, sold quantity, and sold price. The join combines transactional line-item data with product metadata for display.

38. Explain the inferred ER diagram.

Direct answer: `users` stores account identity and role. `products` stores each product's name, price, quantity, and category text. `sales` is the parent transaction with ID, date/time, and total amount. `sale_items` is the junction table that links a sale to one or more products and stores sold quantity and price. `logs` stores username, role, message, and timestamp. One sale has many sale items, and one product can occur in many sale items, so sales and products are many-to-many through sale_items.

```
users(id PK?, username UNIQUE, password_hash, role, ...)
products(id PK, name, price, quantity, category)
sales(id PK, datetime, total_amount, cashier_id FK recommended)
sale_items(sale_id FK, product_id FK, quantity, price)
logs(id PK recommended, username, role, message, timestamp)

sales 1 --- many sale_items many --- 1 products
```

39. Why store price in sale_items if products already has price?

Direct answer: Product price is the current catalog price and can change tomorrow. The `sale_items.price` field records the unit price at the time of sale, so old invoices, refunds, and reports remain historically correct.

40. Is the database normalized?

Direct answer: The design is partly normalized because sales and products are separated and sale_items prevents repeating groups in a sale. However, category is stored as text in products, which can create inconsistent spellings and repeated data. A more normalized design uses a `categories` table with a category foreign key. The exact normal form cannot be proven without the DDL and functional dependencies.

41. How does logging work, and what is inaccurate in the CV?

Direct answer: The project has a custom `LoggerUtil` class that inserts username, role, and message into a MySQL `logs` table. That is database-backed audit logging, not the Java Logging API (`java.util.logging`). In an interview I would describe it accurately as custom JDBC audit logging.

Current code truth: `BillingForm` logs 'Completed a sale' in its constructor before any sale is finalized, so the audit event is inaccurate. Product CRUD is not logged.

Production bar: I would log after a successful commit, include a correlation ID and sale ID, use structured log fields, avoid passwords or sensitive data, and add Java application logging for operational errors separate from business audit logs.

42. What is the difference between authentication and authorization here?

Direct answer: Authentication verifies who a user is, such as username and password. Authorization decides what the authenticated user may do, such as product management for admins only. The current code routes users to different Swing dashboards, but a production system must enforce authorization in the service/data layer as well, because hiding a button is not a security boundary.

43. What database indexes would you add?

Direct answer: At minimum, I would make username unique and indexed, keep primary-key indexes on IDs, index foreign keys such as `sale_items.sale_id` and `sale_items.product_id`, and index `sales.datetime` if sales history is frequently sorted or filtered by date. I would measure slow queries first because indexes speed reads but add write and storage cost.

44. Why should money not use double?

Direct answer: Binary floating-point values cannot represent many decimal fractions exactly, which can produce rounding errors in totals. For currency I would use `BigDecimal` in Java and a fixed decimal type such as `DECIMAL(10,2)` in MySQL, define a rounding policy, and store the final billed amount consistently.

45. What are the main production weaknesses in the supplied code?

Direct answer: The main weaknesses are hard-coded database credentials, plaintext password comparison, no explicit rollback, no conditional stock validation, UI-only role separation, a sale-completion log written before completion, `double` for money, thin input constraints, direct UI-to-SQL coupling, and no evidence of tests or connection pooling. I would acknowledge these as learning-project limitations and explain the specific production fixes.

46. How would you improve the project architecture?

Direct answer: I would use layered design: Swing UI for presentation, services for billing and authorization rules, repositories/DAOs for JDBC, domain models such as Product and Sale, and a configuration/secret layer. This lets me unit-test billing rules without a UI, replace persistence more safely, and prevent SQL from being spread across event handlers.

Supermarket SQL questions with direct answers

47. What is an INNER JOIN?

Direct answer: An INNER JOIN returns rows that match in both tables. Sales history uses one: `sale_items` joins `products` by product ID, so each line item can show its product name.

48. What is a LEFT JOIN?

Direct answer: A LEFT JOIN returns every row from the left table and matching rows from the right table when present. I would use it to find products that have never been sold by left-joining products to sale_items and filtering rows where the sale item is null.

49. WHERE versus HAVING?

Direct answer: WHERE filters rows before grouping; HAVING filters groups after aggregation. For example, WHERE can restrict a date range, while HAVING can return only products whose SUM(quantity) is greater than ten.

50. DELETE versus TRUNCATE versus DROP?

Direct answer: DELETE removes selected rows and can use WHERE. TRUNCATE quickly removes all rows and typically resets storage behavior. DROP removes the table definition itself. I would use DELETE carefully for a product only after considering sale-history foreign keys and audit requirements.

51. What is SQL injection?

Direct answer: It happens when untrusted input changes the structure of a SQL command. PreparedStatement prevents it by sending SQL structure and parameter values separately. It does not replace authorization or input validation.

52. What is a foreign key?

Direct answer: A foreign key references a primary or unique key in another table and enforces referential integrity. For example, `sale_items.sale_id` should reference `sales.id` and `sale_items.product_id` should reference `products.id`.

```
-- 1. Top products by quantity sold
SELECT p.name, SUM(si.quantity) AS units_sold
FROM sale_items si
JOIN products p ON p.id = si.product_id
GROUP BY p.id, p.name
ORDER BY units_sold DESC;

-- 2. Products never sold
SELECT p.id, p.name
FROM products p
LEFT JOIN sale_items si ON si.product_id = p.id
WHERE si.product_id IS NULL;

-- 3. Daily revenue
SELECT DATE(datetime) AS sale_day, SUM(total_amount) AS revenue
FROM sales
GROUP BY DATE(datetime)
ORDER BY sale_day DESC;

-- 4. Sale detail for a selected sale
SELECT si.product_id, p.name, si.quantity, si.price
FROM sale_items si
JOIN products p ON p.id = si.product_id
WHERE si.sale_id = ?;
```

4. Java, OOP, JDBC, DBMS, and SQL fundamentals

53. What are JDK, JRE, and JVM?

Direct answer: The JDK contains development tools such as the compiler plus the runtime. The JRE provides the runtime libraries and JVM needed to run Java applications. The JVM executes bytecode and gives Java platform independence because the same bytecode can run on JVMs for different operating systems.

54. What are the four pillars of OOP?

Direct answer: Encapsulation protects object state through controlled methods. Abstraction exposes useful operations while hiding details. Inheritance lets a subtype reuse or extend a base type. Polymorphism lets code use a shared interface while different objects provide different behavior. In this project, layered billing services and payment interfaces are good design examples; I would not claim inheritance or payment polymorphism exists in the current source unless I add it.

55. Overloading versus overriding?

Direct answer: Overloading uses the same method name with different parameter lists and is resolved at compile time. Overriding supplies a subclass implementation of an inherited method with the same signature and is resolved at runtime.

56. Abstract class versus interface?

Direct answer: An abstract class can hold shared state, constructors, and partial implementation. An interface defines a contract and supports multiple interface implementation. I use an interface when unrelated classes share a capability, such as a PaymentProcessor contract, and an abstract class when related classes share state and behavior.

57. ArrayList versus LinkedList?

Direct answer: ArrayList gives fast random access and amortized append but can require shifting elements for middle inserts. LinkedList makes node insertion/removal cheap once the node is known but has slow indexed access and more memory overhead. For most application code I start with ArrayList unless I have a measured reason not to.

58. HashMap versus HashSet?

Direct answer: HashMap stores key-value pairs; HashSet stores unique values and is commonly backed by a map. Both typically offer O(1) average lookup, but collisions and resizing affect details.

59. Checked versus unchecked exceptions?

Direct answer: Checked exceptions must be declared or handled, such as SQLException. Unchecked exceptions inherit RuntimeException and often indicate programming errors or invalid assumptions. I catch SQLException at an appropriate boundary, preserve useful context, and avoid swallowing it silently.

60. What is try-with-resources?

Direct answer: It automatically closes resources that implement AutoCloseable even if an exception occurs. JDBC Connection, PreparedStatement, and ResultSet should be scoped this way to avoid leaks. The current project uses try-with-resources for several connections but should also include all statements and result sets in the resource declaration.

61. What is a transaction isolation problem?

Direct answer: It occurs when concurrent transactions observe or modify data in a way that causes anomalies such as dirty reads, non-repeatable reads, phantom reads, or lost updates. Inventory needs the database transaction and conditional update to prevent lost updates and overselling.

62. What is normalization?

Direct answer: Normalization organizes relational data to reduce duplication and anomalies. First normal form uses atomic values, second normal form removes partial dependency on part of a composite key, and third normal form removes transitive dependency. I normalize operational data, then denormalize only with a measured reporting reason.

JDBC flow you should explain

```
try (Connection conn = dataSource.getConnection();
     PreparedStatement ps = conn.prepareStatement(SQL)) {
    ps.setString(1, username);
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            // Map row to domain object
        }
    }
}
```

63. Explain the JDBC lifecycle.

Direct answer: Load or configure a JDBC driver, obtain a Connection, prepare a parameterized statement, bind parameters, execute the query or update, read the ResultSet or update count, commit or roll back if a transaction is active, and close resources. In production I obtain connections from a pool rather than creating a raw connection for every operation.

64. What are 1NF, 2NF, and 3NF using sale_items?

Direct answer: For 1NF, each sale-item row stores one product and one quantity rather than a comma-separated product list. For 2NF, if the key is `(sale_id, product_id)`, fields such as product name belong in products because they depend only on product ID. For 3NF, category information should be separated from products if it has its own attributes, so non-key product fields do not depend transitively on a category description.

5. Python-first coding, DSA, and complexity

Python is the primary coding language for this handbook. In a live round, first clarify assumptions, state the approach, write the code, test an edge case, and give time/space complexity. Do not rush into code.

65. Reverse a string

Direct answer: Use slicing for concise Python, but explain that it creates a new string. For interview reasoning, a two-pointer list approach shows the algorithmic idea. Time O(n), extra space O(n) because Python strings are immutable.

```
def reverse_text(s):
    return s[::-1]
```

66. Check a palindrome

Direct answer: Compare characters from both ends. Normalize case or remove non-alphanumeric characters only if the question asks for it. Time O(n), extra space O(1) excluding normalization.

```
def is_palindrome(s):
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1
    return True
```

67. Check two anagrams

Direct answer: Count characters using Counter or a dictionary. Mention whether spaces and case matter. Time O(n), extra space O(k).

```
from collections import Counter

def are_anagrams(a, b):
    return Counter(a.replace(' ', '').lower()) == Counter(b.replace(' ', '').lower())
```

68. Find the second largest distinct value

Direct answer: Track the largest and second largest in one pass. State how duplicate values are handled. Time O(n), space O(1).

```
def second_largest(values):
    first = second = float('-inf')
    for x in values:
        if x > first:
            first, second = x, first
        elif first > x > second:
            second = x
    return None if second == float('-inf') else second
```

69. Find duplicate values

Direct answer: Use a seen set and add an item to duplicates only the first time it repeats. Time O(n), space O(n).

```
def duplicates(values):
    seen, repeated = set(), set()
    for value in values:
        if value in seen:
            repeated.add(value)
        else:
            seen.add(value)
    return repeated
```

70. Find a missing number from 1 to n

Direct answer: Use the expected sum minus actual sum, or XOR if overflow matters in another language. Time O(n), space O(1).

```
def missing_number(values, n):
    return n * (n + 1) // 2 - sum(values)
```

71. Binary search

Direct answer: Binary search requires sorted input. Maintain left and right inclusive bounds and shrink the range. Time O(log n), space O(1).

```
def binary_search(values, target):
    left, right = 0, len(values) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if values[mid] == target:
            return mid
        if values[mid] < target:
            left = mid + 1
```

```
        else:
            right = mid - 1
    return -1
```

72. Balanced parentheses

Direct answer: Use a stack. Push opening brackets; for a closing bracket, pop and check that it matches. Time O(n), space O(n).

```
def balanced(text):
    pairs = {')': '(', ']': '[', '}': '{'}
    stack = []
    for ch in text:
        if ch in '([{':
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack
```

73. First non-repeating character

Direct answer: Count frequencies then make a second pass in original order. Time O(n), space O(k).

```
from collections import Counter

def first_unique(s):
    counts = Counter(s)
    return next((ch for ch in s if counts[ch] == 1), None)
```

74. Fibonacci

Direct answer: Use iteration instead of naive recursion to avoid exponential repeated work. Time O(n), space O(1).

```
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

More prompts - know the direct approach

Prompt	Direct approach	Complexity
Prime check	Test divisors through square root.	O(sqrt n), O(1)
Character frequency	Dictionary / Counter.	O(n), O(k)
Common elements of arrays	Use a set; preserve order only if asked.	O(n + m), O(n)
Maximum subarray sum	Kadane's algorithm.	O(n), O(1)
Two sum	Map value to prior index.	O(n), O(n)
Merge two sorted arrays	Two pointers.	O(n + m), O(n + m)
Reverse linked list	Iterate with prev/current/next pointers.	O(n), O(1)
BFS versus DFS	Queue for BFS; stack/recursion for DFS.	O(V + E)
Valid parentheses	Stack and matching map.	O(n), O(n)
Frequency of words	Split carefully then dictionary count.	O(n), O(k)

75. List, tuple, set, and dictionary?

Direct answer: A list is ordered and mutable. A tuple is ordered and immutable. A set stores unique values with fast average membership checks. A dictionary maps unique keys to values. I choose based on ordering, mutability, and lookup needs.

76. == versus `is` in Python?

Direct answer: == compares value equality; `is` compares object identity. I use `is None` for the singleton None and == for normal value comparison.

77. What is a generator?

Direct answer: A generator yields values lazily, so it can process large sequences without storing all results in memory. A function containing `yield` returns a generator iterator.

78. What are `*args` and `**kwargs`?

Direct answer: `*args` collects positional arguments into a tuple; `**kwargs` collects named arguments into a dictionary. They are useful for flexible wrapper functions but should not hide a clear API unnecessarily.

79. What is a decorator?

Direct answer: A decorator wraps a function or class to add behavior such as logging, timing, authentication, or caching without changing the core function body.

80. Shallow versus deep copy?

Direct answer: A shallow copy creates a new outer container but retains references to nested objects. A deep copy recursively copies nested objects. I choose based on whether nested mutation must be isolated.

6. C, C++, JavaScript, HTML, CSS, Tailwind, and API fundamentals

81. C versus C++?

Direct answer: C is primarily procedural and gives low-level control. C++ extends that environment with classes, RAII, templates, STL, and object-oriented and generic programming. Both require careful memory and resource management.

82. Pointer versus reference in C++?

Direct answer: A pointer stores an address, may be null, can be reassigned, and requires dereferencing. A reference is an alias that must be initialized and is normally not reseated. I prefer references for required non-owning parameters and pointers when absence or reseating is meaningful.

83. Stack versus heap?

Direct answer: Stack memory is automatically managed for function-call frames and has limited lifetime/scope. Heap memory is dynamically allocated and has a longer controlled lifetime; in modern C++ I prefer RAII and smart pointers rather than raw `new` and `delete`.

84. What is a virtual function?

Direct answer: A virtual function enables runtime polymorphism. When I call it through a base pointer or reference, C++ dispatches to the derived override. A pure virtual function makes the class abstract.

85. What is RAII?

Direct answer: Resource Acquisition Is Initialization ties resource lifetime to object lifetime. Constructors acquire resources and destructors release them, so resources are cleaned even when exceptions occur. Smart pointers and file/lock wrappers use this principle.

86. `var`, `let`, and `const` in JavaScript?

Direct answer: Use `const` by default for bindings that are not reassigned, `let` when reassignment is needed, and avoid `var` because its function scope and hoisting behavior are easier to misuse.

87. Promise versus async/await?

Direct answer: A Promise represents a future result. `async/await` is syntax built on promises that makes asynchronous code read more like sequential code while still requiring error handling with try/catch.

88. What is the DOM?

Direct answer: The Document Object Model is the browser's object representation of an HTML document. JavaScript can select, create, update, and listen to events on DOM nodes.

89. What is semantic HTML?

Direct answer: Semantic elements communicate meaning, such as `header`, `nav`, `main`, `article`, `button`, and `label`. They improve accessibility, maintainability, and search understanding compared with using only generic divs.

90. Flexbox versus CSS Grid?

Direct answer: Flexbox is best for one-dimensional layout along a row or column. Grid is best for two-dimensional row-and-column layout. I often combine them: Grid for page structure and Flexbox inside components.

91. What is Tailwind CSS?

Direct answer: Tailwind is a utility-first CSS framework. It provides small composable classes for spacing, typography, colors, responsive rules, and states. It can speed consistent UI work, but I should not claim it was used in the supplied AI reviewer because its CSS is handwritten and no Tailwind configuration exists.

92. What is CORS?

Direct answer: Cross-Origin Resource Sharing is a browser security mechanism controlled by response headers. A backend must explicitly allow trusted origins, methods, and headers when a frontend is hosted on a different origin. I would use a restrictive allowlist rather than * for authenticated services.

93. PUT versus PATCH?

Direct answer: PUT generally replaces the full representation at a known URI and is intended to be idempotent. PATCH applies a partial change. In real APIs I document the contract precisely because semantics matter to clients and retries.

94. Authentication versus authorization?

Direct answer: Authentication answers who the caller is; authorization answers what that authenticated caller may do. Both are needed for a secure product.

7. Cloud, Firebase, Docker, Git, GitHub, AWS, and DevOps

95. What is cloud computing?

Direct answer: Cloud computing provides on-demand compute, storage, networking, and managed services over the internet. I use IaaS for virtual machines, PaaS/serverless platforms for managed deployment, and SaaS for complete applications. The key trade-off is faster delivery and managed operations versus service limits and vendor-specific design.

96. Compute Engine versus Cloud Functions?

Direct answer: Compute Engine gives a managed virtual machine where I control the OS and runtime. Cloud Functions run event-driven or HTTP code without managing servers. I choose Functions for small stateless event/API workloads and VMs when I need OS-level control, long-running processes, or custom runtime behavior.

97. Cloud Storage versus MySQL?

Direct answer: Cloud Storage is object storage for files such as documents or images. MySQL is a relational database for structured records, relationships, transactions, and SQL queries. I would store a file in object storage and keep its metadata, owner, and permissions in a database.

98. What is IAM and least privilege?

Direct answer: IAM defines identities, roles, and permissions. Least privilege means granting each user, service account, or pipeline only the minimum actions and resources needed. It reduces the impact of mistakes or credential compromise.

99. What is a service account?

Direct answer: A service account is a non-human identity for workloads. A deployed function should use a service account with narrowly scoped permissions rather than a developer's personal credential or an exported key file.

100. What is observability?

Direct answer: Observability is the ability to understand system behavior through logs, metrics, traces, and alerts. For the code reviewer I would monitor request count, latency, error rate, model failures, input/output size, and cost, while avoiding raw proprietary code in logs.

101. What is Docker?

Direct answer: Docker packages an application and dependencies into an image. A container is a running instance of that image. It improves environment consistency, but it does not replace secure configuration, orchestration, monitoring, or testing.

102. Image versus container?

Direct answer: An image is an immutable blueprint containing application files and runtime layers. A container is a running isolated process created from that image.

103. What is a Dockerfile?

Direct answer: A Dockerfile is a recipe for building an image. Typical instructions are FROM, WORKDIR, COPY, RUN, EXPOSE, and CMD. I use a .dockerignore to avoid copying node_modules, secrets, and unnecessary files into the build context.

104. Git versus GitHub?

Direct answer: Git is distributed version-control software. GitHub is a hosted collaboration platform around Git that adds pull requests, reviews, Actions, issue tracking, and repository hosting.

105. What is a pull request?

Direct answer: A pull request proposes a branch change for review before merging. A good pull request has a focused scope, clear description, tests, and no secrets or unrelated formatting changes.

106. What is CI versus CD?

Direct answer: Continuous Integration automatically builds and tests changes frequently. Continuous Delivery keeps releases deployable and may require a human approval. Continuous Deployment releases validated changes automatically. I would use CI for every pull request and a controlled delivery pipeline for cloud deployment.

107. What is a safe CI/CD pipeline for the AI project?

Direct answer: Checkout code, install locked dependencies, lint and test, scan dependencies/secrets, build the function, deploy to staging using a least-privilege workload identity, run smoke tests, require approval for production, deploy, monitor errors and latency, and retain a rollback path.

108. AWS and GCP equivalents?

Direct answer: At a high level, GCP Compute Engine corresponds to AWS EC2, Cloud Storage to S3, Cloud Functions to Lambda, and Cloud Monitoring/Logging to CloudWatch. The concepts overlap, but IAM models, service limits, deployment tooling, and operational details differ.

109. What is a secret manager?

Direct answer: It stores sensitive configuration such as API keys and database passwords outside source control. Workloads retrieve secrets at runtime through controlled identity permissions. It supports rotation and auditability better than hard-coded strings or `.env` files copied into production.

SDLC, testing, and DevOps flow

```
Plan -> requirements -> design -> implement -> code review -> unit tests
-> integration tests -> security/dependency checks -> build artifact
-> staging deploy -> smoke tests -> approval -> production deploy
-> monitor logs/metrics/alerts -> incident response -> retrospective
```

110. What are the SDLC phases?

Direct answer: A practical SDLC includes requirements, design, implementation, testing, deployment, operations, and maintenance. Agile teams repeat these in small increments rather than completing all design before all implementation.

111. Agile versus Waterfall?

Direct answer: Waterfall progresses through largely sequential phases and works best when requirements are stable. Agile delivers small increments, collects feedback, and adapts. I choose based on uncertainty, compliance needs, and the cost of change rather than treating one as universally better.

112. Unit versus integration versus system testing?

Direct answer: Unit tests isolate a small function or class. Integration tests verify components together, such as a function plus a test database/API. System or end-to-end tests exercise the full user workflow. A healthy test strategy has many fast unit tests and fewer slower end-to-end tests.

113. What is regression testing?

Direct answer: Regression testing checks that a previously working behavior still works after change. I add a regression test whenever I fix an important bug so the same defect does not return.

114. What is a rollback plan?

Direct answer: It is a prepared way to restore a known-good version when deployment harms users. It includes versioned artifacts, reversible database changes where possible, health checks, ownership, and decision criteria. For serverless code, I would keep the prior release available and monitor error/latency after rollout.

8. OS, networking, DBMS, and out-of-nowhere fundamentals

115. Process versus thread?

Direct answer: A process has its own address space and system resources. Threads are execution units inside a process and share its memory. Threads are lighter but require synchronization to avoid race conditions.

116. What is a race condition?

Direct answer: It occurs when the result depends on timing of concurrent operations on shared state. Two cashiers decrementing inventory is a business example. I prevent it with database transactions, conditional updates, constraints, and appropriate isolation rather than trusting UI timing.

117. What is deadlock?

Direct answer: Deadlock occurs when transactions wait in a cycle for resources held by one another. I reduce it by keeping transactions short, accessing tables/rows in a consistent order, indexing queries, and retrying carefully when the database reports a deadlock.

118. What is virtual memory?

Direct answer: Virtual memory gives each process an address-space abstraction and lets the OS map pages to physical memory or disk. It improves isolation and allows programs to use more address space than immediately available RAM, though excessive paging hurts performance.

119. TCP versus UDP?

Direct answer: TCP is connection-oriented, reliable, ordered, and congestion-controlled. UDP is connectionless with lower overhead but no built-in delivery or ordering guarantee. I use TCP/HTTPS for APIs where correctness matters; UDP suits latency-sensitive cases that can tolerate loss.

120. HTTP versus HTTPS?

Direct answer: HTTPS is HTTP protected by TLS. TLS encrypts data in transit, authenticates the server through certificates, and detects tampering. API keys, passwords, and user data should always travel over HTTPS.

121. What happens when a browser calls an HTTPS API?

Direct answer: DNS resolves the hostname, a TCP connection is established, TLS negotiates encryption and validates the certificate, then the browser sends an HTTP request. The server responds with an HTTP status, headers, and body. In modern deployments, caches, load balancers, and serverless routing may sit in between.

122. Common HTTP status codes?

Direct answer: 200 means successful request, 201 resource created, 400 invalid client request, 401 unauthenticated, 403 authenticated but forbidden, 404 not found, 409 conflict, 429 rate limited, and 500 unexpected server failure. I choose codes that match the actual client action and provide safe error details.

123. What is DNS?

Direct answer: Domain Name System translates domain names to IP addresses and related records. It is distributed and cached, which is why DNS changes can take time to propagate.

124. What is Big O?

Direct answer: Big O describes how time or memory grows as input size grows. It guides algorithm choice: O(log n) binary search scales better than O(n) linear search on sorted data, and O(n^2) nested loops often become expensive at large input.

125. Stack versus queue?

Direct answer: A stack is last-in first-out and fits matching parentheses or undo. A queue is first-in first-out and fits breadth-first search and request scheduling.

126. BFS versus DFS?

Direct answer: BFS explores level by level with a queue and finds shortest path in an unweighted graph. DFS explores deeply with a stack or recursion and is useful for traversal, connectivity, and cycle-related tasks. Both are O(V + E).

9. HR, behavioral, and mock-interview practice

127. Tell me about yourself.

Direct answer: I am Visvatharsan Saravanan, a Computer Science and Engineering student focused on Java, databases, cloud, and practical software projects. I built a Java Swing and MySQL supermarket application with billing, inventory, JDBC, and audit logging, and a cloud-hosted AI code reviewer using Firebase Functions and Gemini. These projects taught me to connect UI, backend logic, databases, APIs, and cloud deployment. I enjoy consistent problem solving, reflected in my 50-day LeetCode streak, and I want to build strong delivery skills on real client work at Capgemini.

128. Why Capgemini?

Direct answer: Capgemini appeals to me because it works on real client transformation across cloud, applications, data, and digital engineering. My projects have given me a foundation in Java, MySQL, APIs, and cloud services, and I want to deepen that foundation in a disciplined delivery environment. I value learning from experienced engineers while contributing reliably to a team and customer outcomes.

129. Why should we hire you?

Direct answer: I bring practical evidence of learning: I can explain two end-to-end applications, their data flow, their limitations, and how I would improve them for production. I am comfortable with Java, SQL, cloud fundamentals, and Python problem solving. I do not overstate my work; I learn quickly, communicate clearly, and can take ownership of a defined task.

130. What is your strength?

Direct answer: My strength is structured problem solving. I break a feature into user flow, data flow, failure cases, and validation rather than treating it as one large problem. In the supermarket application, that meant separating product lookup, cart calculation, sale persistence, and inventory updates; in the AI reviewer, it meant separating browser input, backend secret handling, model call, and result display.

131. What is your weakness?

Direct answer: Earlier, I spent too long trying to resolve every issue independently before asking for feedback. I am improving by time-boxing investigation, writing down what I tried, and asking focused questions when that will unblock the task. This helps me balance ownership with delivery speed.

132. Tell me about a project challenge.

Direct answer: In the billing flow, the challenge was making one user action update several related records safely: a sale header, sale items, and product stock. I learned that this must be treated as one transaction. My current learning-project version commits the batch, and my production improvement is explicit rollback plus a conditional stock update so a partial sale or negative stock cannot occur.

133. How do you handle an unknown technology?

Direct answer: I first define the smallest outcome I need, read official documentation, build a narrow proof of concept, test the happy path and failure path, then integrate it behind a clear boundary. For example, with the Gemini integration I learned the request/response flow, kept the key server-side through Firebase secrets, and designed the response contract for the UI.

134. Are you open to relocation and shifts?

Direct answer: Yes. I understand client delivery can require flexibility in location, team schedule, and learning new tools. I am prepared to discuss the role's expectations and adapt professionally.

30-minute mock interview

Minute	Interviewer prompt	Your focus
0-2	Tell me about yourself.	Use the direct answer above; do not list every skill.
2-8	Explain AI Code Reviewer.	Draw hosting -> function -> Gemini; name current limits honestly.
8-15	Explain supermarket billing.	Draw sale, sale_items, product relation; explain transaction and oversell fix.
15-21	Write two SQL queries and explain OOP.	PreparedStatement, join, GROUP BY/HAVING, four pillars.
21-25	Cloud/DevOps follow-up.	Secrets, IAM, CI/CD recommendation, testing and monitoring.
25-28	Python coding prompt.	Clarify, explain, code, test, complexity.
28-30	Why Capgemini / relocation?	Be calm, concise, and truthful.

10. Coverage matrix and sources

CV skill / evidence	Where covered
Java, Swing, JDBC, MySQL	Chapters 3 and 4
C++, C	Chapter 6
Python	Chapter 5
HTML, CSS, JavaScript	Chapters 2 and 6
GCP, Firebase, Gemini API	Chapters 2 and 7
Git, GitHub, Docker	Chapters 2 and 7
AWS	Chapter 7
SQL, normalization, ER diagram	Chapters 3 and 4
SDLC, DevOps, testing	Chapter 7
OS, networking, DSA	Chapters 5 and 8
Behavioral, communication, adaptability	Chapter 9

Primary documentation used for industry-grade recommendations

- Firebase Functions HTTPS v2 reference: https://firebase.google.com/docs/reference/functions/2nd-gen/node/firebase-functions.https
- Firebase environment and secrets: https://firebase.google.com/docs/functions/config-env
- Firebase Functions v2 migration and secret example: https://firebase.google.com/docs/functions/2nd-gen-upgrade
- Gemini structured outputs and validation: https://ai.google.dev/gemini-api/docs/structured-output
- Gemini Generate Content API: https://ai.google.dev/gemini-api/docs/generate-content
- MySQL InnoDB transaction management: https://dev.mysql.com/doc/refman/en/optimizing-innodb-transaction-management.html
- Oracle JDBC Developer's Guide: https://docs.oracle.com/cd/F39414_01/jjdbc/jdbc-developers-guide.pdf
- OWASP Password Storage Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OWASP Cross Site Scripting Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- Docker overview: https://docs.docker.com/get-started/docker-overview/
- GitHub Actions documentation: https://docs.github.com/actions

Recent candidate-reported interview context - non-official

- August 2026 Exceller discussion: https://www.reddit.com/r/Capgemini_india/comments/1vwu03m/placed_in_capgemini_via_exceller_drive_need_real/
- September 2026 technical-round discussion: https://www.reddit.com/r/Capgemini_india/comments/1wsk7rb/capgemini_exceller_202627_oncampus_how_was_the/
- October 2026 first-interview discussion: https://www.reddit.com/r/Capgemini_india/comments/1wzvrlm/capgemini_exceller_2027_first_interview_experience/

Final rule: your CV is a promise. Explain only what you implemented, state limitations confidently, and show that you know the production-quality next step.